

Name: Multiply By Term

Input:

- **coefficient**: a number in Z256 representing the coefficient of the term
- **power**: an integer representing the power of the term
- **polynomial**: an instance of the class Polynomial that will be multiplied by the term represented with coefficient **coefficient** and exponent **power**

Output:

- A new Polynomial that is the product of multiplying this polynomial by (coefficient) * $x^{(power)}$.
1. **result_terms** $\leftarrow$ an empty map
 2. for each **old_power**, **old_coefficient** in the set of keys and values in **polynomial** do
 - a. **new_pow** $\leftarrow$ **old_power** + **power**
 - b. **new_coeff** $\leftarrow$ the product of **old_coefficient** and **coefficient** using z256 math
 - c. add the new correspondence **new_pow** $\mapsto$ **new_coeff** to the map **result_terms**
 3. Return a new instance of the polynomial class with field **result_terms**

Name: Polynomial Addition

Input:

- polynomial**: an instance of the class Polynomial
- other_polynomial**: a Polynomial object that **polynomial** will be added to

Output:

- a new Polynomial that is the sum of both polynomials.
1. **sum_poly** $\leftarrow$ a copy of the Polynomial object **polynomial**
 2. for each **powers**, **coeffs** in the keys and values of **other_polynomial** do
 - a. **sum_poly** $\leftarrow$ the Polynomial returned by *add_term*(**coeffs**, **powers**) on the object **sum_poly**
 3. return **sum_poly**

Name: Polynomial Multiplication

Input:

- polynomial**: an instance of the class Polynomial
- other_polynomial**: a Polynomial object that **polynomial** will be multiplied to

Output:

- a new Polynomial that is the product of both polynomials.
1. **result** $\leftarrow$ an Polynomial object representing the polynomial 0
 2. for each **pow**, **coeff** in the keys and values of **other_polynomial** do
 - a. **partial_product** $\leftarrow$ the Polynomial returned by *multiply_by_term*(**coeff**, **pow**) on the object **polynomial**

- b. **result** $\leftarrow$ the Polynomial returned by *add_polynomial*(**partial_product**) on the object **result**
3. return **result**

Name: Polynomial Remainder

Input:

- **denominator**: a Polynomial object
- **polynomial**: an instance of the class Polynomial that will be divided by **denominator**

Output

- a new polynomial that is the remainder
1. **remainder** $\leftarrow$ a copy of the Polynomial object **polynomial**
 2. while the value returned by *get_degree*() on the object **remainder** is less than or equal to the value returned by *get_degree*() on the object **denominator** do
 - a. **n_power** $\leftarrow$ the value returned by *get_degree*() on the object **remainder**
 - b. **d_power** $\leftarrow$ the value returned by *get_degree*() on the object **denominator**
 - c. **denominator**
 - d. **n_coeff** $\leftarrow$ the value returned by *get_coefficient*(**n_power**) on the object **remainder**
 - e. **d_coeff** $\leftarrow$ the value returned by *get_coefficient*(**d_power**) on the object **denominator**
 - f. if the value returned by *get_coefficient*(**n_power**) on the object **remainder** is equal to 0 then
 - i. return **remainder**
 - g. **quotient_term** $\leftarrow$ the Polynomial returned by *divide_terms*(**n_coeff**, **n_power**, **d_coeff**, **d_power**)
 - h. **sub_poly** $\leftarrow$ the Polynomial returned by *multiply_by_polynomial*(**quotient_term**) on the object **denominator**
 - i. **remainder** $\leftarrow$ the Polynomial returned by *subtract_polynomial*(**sub_poly**) on the object **remainder**
 - j. the object **remainder**
 3. return **remainder**

6. Discussion

1. If you are given a message that you want to encode and a value of k , which indicates how many error correction bytes you need, is it possible to guarantee that you will not have any coefficients that are equal to zero in the remainder from dividing the message polynomial by the generator polynomial? If there were coefficients that are equal to zero in the encoded data, would it be a problem? Why or why not?

It is **not** possible to guarantee that you will have no coefficients that are equivalent to zero in $\mathbb{Z}_{256}$ in the remainder. This is due to the fact that when you divide the message polynomial by the generator polynomial, this operation is performed using $\mathbb{Z}_{256}$ math for the coefficients. Since it is possible to get zero in $\mathbb{Z}_{256}$, it would not be a problem if there were coefficients equivalent to 0 in the encoded data, as it would still represent an error correction byte.

2. We have discussed the importance of modularity and writing your recipes/code in such a way that you can reuse them. If you needed a `Polynomial` class to represent polynomials with regular, real-number coefficients (as opposed to coefficients that are elements of $\mathbb{Z}_{256}$), how could you minimally change the code you have already written in order to reuse it for this purpose?

There is an easy way to minimally change the code that I have already written. I would go to the $\mathbb{Z}_{256}$ API and change the math operations from $\mathbb{Z}_{256}$ to standard integer operations. Note that if the $\mathbb{Z}_{256}$ had addition and subtraction implemented in the same way, since they output the same result in $\mathbb{Z}_{256}$, this will be need to be changed for regular integer operations since it no longer holds. You could also just replace the `z256` module with a standard integer math module, minimizing the code you have to change even more.